UninoVet

Clinica veterinaria — aplicacao web, mobile e banco de dados

Tudo sobe junto com um unico docker compose up
App <- consome /api/agendamentos -> Painel <- JDBC -> Banco

Documentacao tecnica · arquitetura, rotas, banco e escolhas do projeto

1. Visao geral

O UninoVet e a aplicacao de uma clinica veterinaria, dividida em tres partes que conversam entre si e sobem juntas em containers. Foi pensada para ser facil de rodar (so precisa de Docker) e facil de evoluir.

- **Banco de dados (MySQL):** guarda tutores, animais, veterinarios e consultas. As tabelas e alguns dados ja sao criados sozinhos pelo arquivo `db/init.sql`.
- **Painel web (Java · JSP/Servlet):** onde a clinica cadastra e gerencia os dados. Tambem expoe uma API em JSON com os agendamentos.
- **App (Flutter):** o foco do projeto. Mostra os agendamentos de forma amigavel, consumindo a API do painel. Ja esta preparado para Android e para web.

> **Integracao entre as partes**
>
> O app nao fala direto com o banco: ele chama a API do painel Java (`/api/agendamentos`), que por sua vez le o MySQL por JDBC. Assim as tres disciplinas (mobile, web/orientacao a objetos e banco) aparecem conectadas.

2. Arquitetura e containers

Tudo sobe junto com um unico docker compose up
App <- consome /api/agendamentos -> Painel <- JDBC -> Banco

Cada parte roda no seu proprio container, orquestrados por um unico `docker-compose.yml`:

- **db** — imagem `mysql:8.0`; roda o `init.sql` na primeira subida.
- **web** — imagem baseada no `tomcat:9`; compila os servlets no proprio build.
- **mobile** — imagem do Flutter; serve o app em modo web na porta 5000.

O `web` so inicia depois que o banco esta saudavel (`healthcheck`), e o `mobile` depois do web.

3. Como rodar

Na pasta `uninoVet`, com o Docker aberto:

```
docker compose up
```

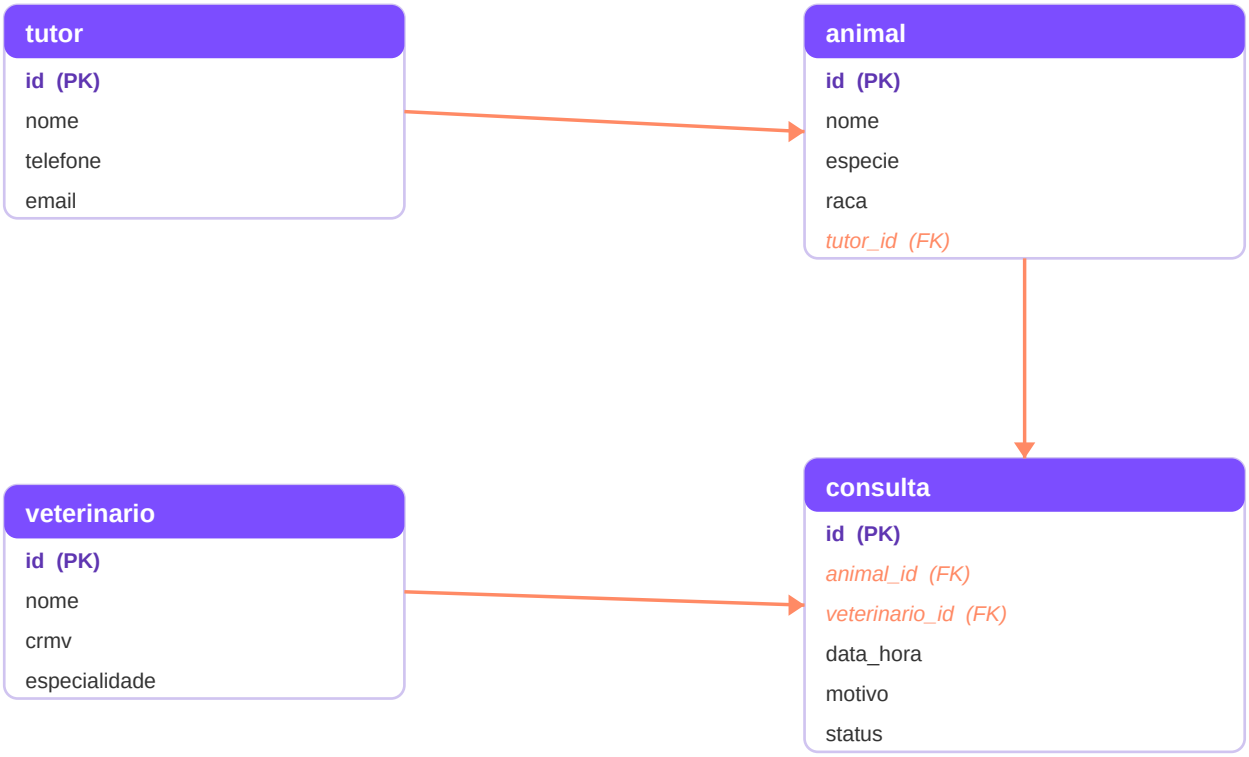
- App (Flutter): `http://localhost:5000`
- Painel (web): `http://localhost:8080`
- Parar tudo: `docker compose down`
- Rodar os testes do app: `docker compose exec mobile flutter test`

Codespace: o arquivo `.devcontainer/devcontainer.json` ja abre as portas 5000, 8080 e 3306 automaticamente ao abrir o repositorio no Codespace.

Android: o projeto do app ja vem configurado para Android (`org br.uninovet`), entao compila um APK com `flutter build apk` dentro do container.

4. Banco de dados

Modelo relacional com quatro tabelas. Um tutor tem varios animais; uma consulta pertence a um animal e a um veterinario.



PK = chave primaria · FK = chave estrangeira (laranja)

Relacionamentos (chaves estrangeiras):

- `animal.tutor_id` → `tutor.id`
- `consulta.animal_id` → `animal.id`
- `consulta.veterinario_id` → `veterinario.id`
- `consulta.status` e um ENUM: AGENDADA, CONCLUIDA ou CANCELADA.

5. Rotas

Painel web (Java)

Rota	O que faz
`GET /`	Menu dos cadastros (tutores, veterinarios, animais, consultas).
`GET /crud?tabela=T`	Lista os registros da tabela T.
`GET /crud?tabela=T&acao;=form`	Formulario para incluir um registro.
`GET /crud?tabela=T&acao;=form&id;=N`	Formulario preenchido para editar o registro N.
`POST /crud`	Salva (inclui ou altera, conforme vier o id).
`POST /crud (acao=excluir)`	Exclui o registro informado.
`GET /api/agendamentos`	Devolve os agendamentos em JSON (consumido pelo app).

Um unico servlet (`CrudServlet`) atende todas as tabelas: a tabela vem no parametro `tabela` e a operacao no `acao`. Isso mantem o codigo Java bem curto.

App (Flutter) — navegacao

Tela / acao	O que faz
`PaginaAgendamentos`	Tela inicial: cabecalho com a patinha, filtros por situacao e a lista de cartoes.
`Filtros`	Todos · Agendadas · Concluidas · Canceladas.
`Detalhe (bottom sheet)`	Ao tocar num cartao, abre os detalhes da consulta.
`Pull-to-refresh`	Puxar a lista para baixo recarrega da API.

O app busca os dados em `Api.buscarAgendamentos()`, que chama `GET {Ambiente.apiBase}/api/agendamentos`. O endereco fica em `lib/config/ambiente.dart`.

6. Escolhas tecnicas

- **MySQL** com o banco todo criado por `init.sql` — sem ORM e sem migracoes, entao o Java fica responsavel so por ler e gravar.
- **Java minimo (JSP/Servlet no Tomcat):** um servlet generico de CRUD (classes `Banco` e `Tabela`) cobre as quatro tabelas, e um servlet so para a API. O foco de codigo fica no app.
- **Integracao por API JSON** (banco → Java → Flutter), com CORS liberado, para as tres disciplinas aparecerem conectadas.
- **Flutter e o foco:** interface mais elaborada e amigavel, com tema trocavel e o logo de patinha desenhado em codigo (`CustomPainter`), sem depender de imagem externa.
- **Tudo configuravel:** textos, tema e endereco da API ficam em `lib/config/`; trocar de negocio e mexer so ali e no banco.
- **Tudo em container:** nao precisa instalar Java, Flutter nem MySQL na maquina — so Docker (ou um Codespace).

7. Estrutura de pastas

```
uninoVet/
  docker-compose.yml       sobe os tres servicos
  .devcontainer/           Codespace (abre as portas)
  db/
    init.sql               tabelas + dados de exemplo
  web/                     painel em Java
    Dockerfile
    src/br/uninovet/       Banco, Tabela, CrudServlet, ApiAgendamentos
    webapp/                index.jsp, crud.jsp, estilo.css
  mobile/                  app Flutter (foco)
    Dockerfile.dev
    lib/
      config/              ambiente, textos, tema
      modelos/             agendamento
      servicos/            api
      paginas/             pagina_agendamentos
      widgets/             patinha, cartao_agendamento
    android/  web/         plataformas
  README.md
```

8. Proximos passos (ideias)

O projeto foi feito para crescer. Alguns caminhos naturais:

- Deixar o app **criar e cancelar** agendamentos (novo endpoint POST no painel).
- No formulario web, trocar os campos de id por **listas de selecao** (escolher o tutor pelo nome).
- Ajustar o endereco da API para funcionar tambem no **Codespace e em producao**.
- Adicionar **login** no painel administrativo.
- Quando virar outro negocio, revisar `init.sql` e a pasta `lib/config/`.

Documento gerado para acompanhar o projeto — sera atualizado conforme ele evoluir.